How Rouge Works: Building Epoch from Scratch

Rouge is an autonomous product development system. It takes a product idea, refines it through interactive Slack conversation, then runs an iterative build-evaluate-improve loop until the product meets a quality bar. No human writes code. No human triages bugs. The system builds, tests, evaluates, and ships — pausing only when it needs a product decision it cannot make itself.

Let's watch it build a product from scratch.

The Stack

Rouge is a Node.js orchestrator that drives Claude via the Anthropic API to build, test, and evaluate web products. Every product it ships comes with a full production stack, provisioned automatically.

How it runs

A Node.js launcher orchestrates the Karpathy Loop. For each phase, it calls the Claude API with a phase-specific system prompt, the project context, and the working directory. Claude reads the codebase, writes code, runs tests, and commits — all through API tool use. The launcher monitors progress via a progress-based watchdog, handles rate limits with smart backoff, manages state transitions, and advances to the next phase.

The Slack bot (Bolt.js in Socket Mode) provides the human control plane: seeding new products, monitoring progress, pausing/resuming builds, approving final reviews, and sending feedback.

What gets deployed

Every Rouge-built product deploys to this stack:

Layer	Tool	Provisioned by
Hosting	Cloudflare Workers (via OpenNext)	Rouge — automatic
Database	Supabase (Postgres + Auth + Realtime)	Rouge — automatic (slot rotation for free tier)
Payments	Stripe	Rouge — sandbox auth, human activates production
Error monitoring	Sentry	Rouge — auto-creates project, writes DSN to env
Product analytics	PostHog (EU hosting)	Rouge — shared project, product-tagged events
Session recording	PostHog	Included with analytics
CI/CD	GitHub Actions (lint, type check, test, build)	Template — ships with project
Security headers	Next.js middleware (CSP, HSTS, X-Frame-Options)	Template — ships with project
i18n	next-intl (key-based, Claude-translated)	Template — ships with project

Legal pages	Privacy policy + Terms of Service scaffolds	Template — ships with project
Browser QA	GStack (headless browser for automated testing)	Rouge — used during product-walk phase

What Rouge manages automatically

- **Deployment**: build → deploy with post-deploy health check and automatic rollback on failure
- **Database migrations**: dry-run preview, destructive operation detection (DROP/TRUNCATE blocked), audit trail
- **State snapshots**: before each phase, state is snapshotted for corruption recovery
- **Cost tracking**: self-calibrating cost estimation from actual phase timings
- **Cross-product learning**: after each completed product, learnings feed into a personal Library that calibrates future builds

Slack integration

Rouge is controlled via Slack. The bot runs in Socket Mode (no public URL needed) and provides:

- **Seeding**: `@Rouge` in a channel starts a seeding conversation. The 8-persona swarm runs through Slack threading — each discipline posts in the thread, and the human responds in the same thread.
- **Monitoring**: The App Home dashboard shows every project with its current state, cycle number, health score, confidence level, staging URL, and feature area progress. Buttons for Start, Pause, Resume, Skip, and Ship It.
- **Notifications**: Phase transitions, QA results, screenshots, and escalations post as rich Block Kit messages with context.
- **Control**: `/rouge start`, `/rouge pause`, `/rouge resume`, `/rouge ship`, `/rouge feedback` — all from Slack. No SSH, no terminal, no deployment dashboard.

- **Final review**: When a product reaches final-review, the human gets a Slack notification with the staging URL. They test at their own pace and either `/rouge ship` to approve or `/rouge feedback` to request changes.

The launcher and Slack bot are separate processes. The launcher writes state; the bot reads it and sends notifications. Human commands write to state, and the launcher picks up changes on the next iteration.

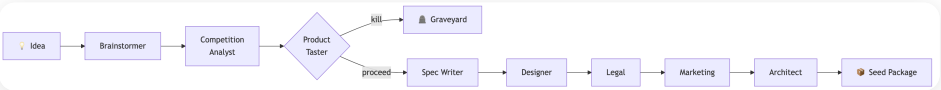
The Idea

The product: **Epoch** — a focus timer that's beautiful enough to leave on screen. A Pomodoro timer for knowledge workers who have tried every timer app and found them either ugly or bloated.

The idea entered Rouge through **Rouge Spec**, the interactive seeding phase. This is where a human and AI co-design a product through Slack conversation — not a requirements document, but a dialogue.

How Rouge Spec Works

Rouge Spec runs a **seeding swarm**: eight discipline-specific AI personas that each interrogate the idea from a different angle. They don't run simultaneously — they take turns, building on each other's output:



The **human participates throughout** — answering questions, making taste decisions, vetoing bad ideas at any stage. The swarm asks; the human decides. The Product Taster is the hard gate: ideas that aren't worth building go to the graveyard before any specs are written.

The eight personas in order:

#	Persona	Role
1	**Brainstormer**	Expand the idea — 10-star version, emotional core, target user
2	**Competition Analyst**	Map the landscape — what exists, where's the gap
3	**Product Taster**	Challenge the idea — kill it or scope it
4	**Spec Writer**	Convert vision into feature areas + acceptance criteria
5	**Designer**	Three-pass: UX architecture → components → visual design
6	**Legal Advisor**	Privacy, terms, compliance
7	**Marketing Strategist**	Positioning, landing page, launch strategy
8	**Technical Architect**	Stack selection, infrastructure, deployment target

When the conversation converges, the seed package contains:

- **Vision document** — emotional north star, aesthetic direction, anti-goals
- **Feature area specifications** — with acceptance criteria per area
- **Design artifact** — style tokens, screen mappings, interaction specs
- **Infrastructure plan** — deployment target, database needs, auth requirements

For Epoch, this conversation established:

- **Emotional north star**: "From 'where did the last 3 hours go?' to 'I can see my focus happening.'"
- **Aesthetic**: Techno-futuristic. Frosted glass, subtle glows, precise typography. Linear's design language meets a high-end watch

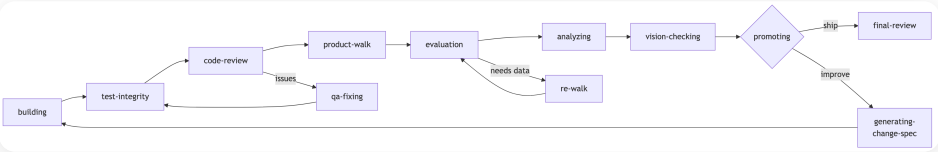
face.

- **Scope**: Full Pomodoro cycle (focus, short break, long break), configurable settings, session counter, keyboard shortcuts, synthesized audio chime. No accounts, no history beyond today, no themes, no task lists.
- **Differentiator**: Optimizes for *presence* — how it feels on your screen — not for features. The design IS the product.

The seed package contained six feature areas with 37 acceptance criteria, a complete design system, and a deployment target. The system estimated one cycle for the initial build. It took five.

The Karpathy Loop

Rouge builds products through an iterative loop inspired by Andrej Karpathy's description of how neural networks train: build, evaluate, adjust, repeat. Each cycle advances the product toward convergence with the vision.



Each phase has a distinct role:

Phase	Purpose	Uses Browser?
building	Write code, run tests, deploy to staging	No
test-integrity	Verify all tests pass, no regressions	No

code-review	Engineering lens — architecture, dead code, security	No
product-walk	Pure observation — screenshot every state, record what exists	Yes
evaluation	Three judges score the walk data (QA, Design, PO lenses)	No
analyzing	Decide: ship, improve, or escalate	No
vision-checking	Compare built product against original vision	No
promoting	Advance to next cycle or trigger final review	No
final-review	Holistic end-of-project walkthrough	Yes

The critical design choice: the browser opens exactly once per cycle (product-walk), and all three evaluation lenses judge the same observation data. This is the **observe-once, judge-through-lenses** architecture.

Building Epoch

Cycle 1: The First Build

Rouge built all six feature areas in a single cycle:

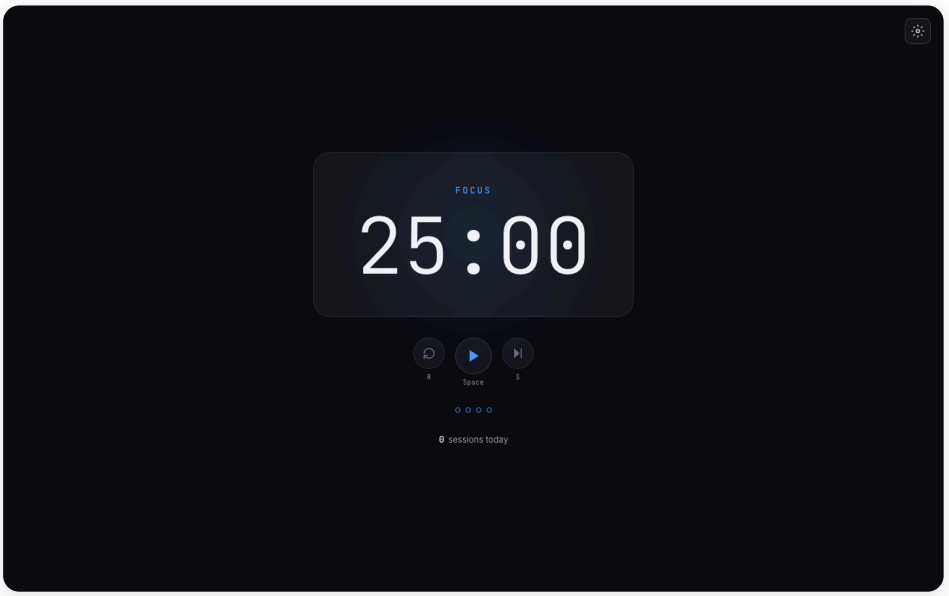
1. **Pomodoro engine** — State machine for focus/short-break/long-break cycling, localStorage persistence, Web Audio API chime synthesis
2. **Timer display** — Responsive monospace digits (72-120px via CSS clamp), phase-colored frosted glass card

3. **Timer controls** — Play/pause, reset, skip with keyboard shortcuts (Space, R, S)
4. **Cycle indicator** — Four dots showing position in the Pomodoro cycle
5. **Session counter** — Daily focus session count, persisted in localStorage
6. **Settings modal** — Eight configurable parameters (durations, auto-start, sound, notifications)

The builder made architectural decisions along the way:

- **Web Audio API** over audio files for the chime — zero dependencies, matches the futuristic aesthetic
- **CSS Modules** over Tailwind — natural syntax for complex effects (backdrop-filter, radial gradients, keyframe animations)
- **Date.now()-based timing** over setInterval — immune to background tab throttling
- **Static export** — the entire app is client-rendered, no SSR needed

Result: 89 tests passing. Deployed to Cloudflare Workers.



Cycle 2: The Quality Ratchet

The evaluation phases identified concrete issues and the builder fixed them:

- Missing error boundary → ErrorBoundary with styled dark fallback UI
- div overlay for settings → Native `<dialog>` element (automatic focus trap, Escape handling)
- Poor contrast (3.2:1) → Fixed to WCAG AA compliance
- No semantic landmarks → HTML5 `<header>`, `<main>`, `<footer>`
- Undiscoverable keyboard shortcuts → Inline hints below controls

Health score jumped from 68 to 82. Accessibility went from 89 to **100**.

Cycles 3–5: Convergence

Each subsequent cycle refined the product further — polishing interactions, improving code architecture, and converging on the vision. The system tracked confidence across cycles and stopped when quality metrics stabilised.

What the System Sees

Rouge does not evaluate products through vibes. Every judgment is structured data with evidence.

Spec criterion check

Each acceptance criterion is evaluated with a pass/fail status and specific evidence:

```
{
  "id": "AC-display-1",
  "criterion": "Timer displays MM:SS in monospace font
```

```
  "status": "pass",
  "evidence": "Timer uses clamp(72px, 15vw, 120px) — mi
}
```

67 of these are checked every cycle. Zero are skipped.

QA health score

Category	Score	Detail
Criteria compliance	100%	67/67 pass
Functional correctness	Clean	0 console errors, all controls verified
Code quality (AI audit)	90/100	Architecture 88, Consistency 92, Robustness 84, Security 96
Final health score	**82**	

PO review — journey quality

Journey	Clarity	Feedback	Efficiency	Delight	Avg
First-time landing	9	8	10	8	9.0
Start/pause focus	9	9	10	8	8.83
Phase transition	9	9	9	9	8.88
Reset current phase	7	9	10	6	8.0

Configure settings	—	—	—	—	8.44
Track daily progress	—	—	—	—	8.5

Overall PO confidence: **0.89**. Verdict: **PRODUCTION_READY**.

QA Tooling

Rouge runs real tools against real deployed code and feeds structured results into the evaluation.

Browser testing — GStack

GStack is a headless Chromium browser controlled via a single-binary CLI. During the product-walk phase, Claude drives it to navigate every screen, click every element, fill forms, test keyboard navigation, and capture screenshots:

```
$B goto https://countdowntimer-staging.gregj64.workers.dev
$B click button:has-text("Start")
$B screenshot screenshots/cycle-3/timer-running.png
$B eval document.querySelectorAll('[aria-label]').length
```

Performance — Lighthouse

GStack runs Lighthouse audits on key pages, tracked across cycles:

Metric	Cycle 1	Cycle 5
Performance	91	91

Accessibility	89	**100**
Best Practices	96	**100**
SEO	100	100

Static analysis

Tool	What it checks
ESLint	Lint errors and warnings
jscpd	Code duplication percentage
madge	Circular dependency detection
knip	Dead code and unused exports
npm audit	Known dependency vulnerabilities
AI code audit	Seven-dimension review: architecture, consistency, robustness, production risks, security, dead/hallucinated code, tech debt

Accessibility

Tested through three channels:

1. **Lighthouse a11y score** — automated WCAG checks
2. **GStack keyboard testing** — Tab through every element, verify Enter/Space activation, check focus trapping in modals
3. **Accessibility tree inspection** — captures landmarks, heading hierarchy, ARIA labels, and roles

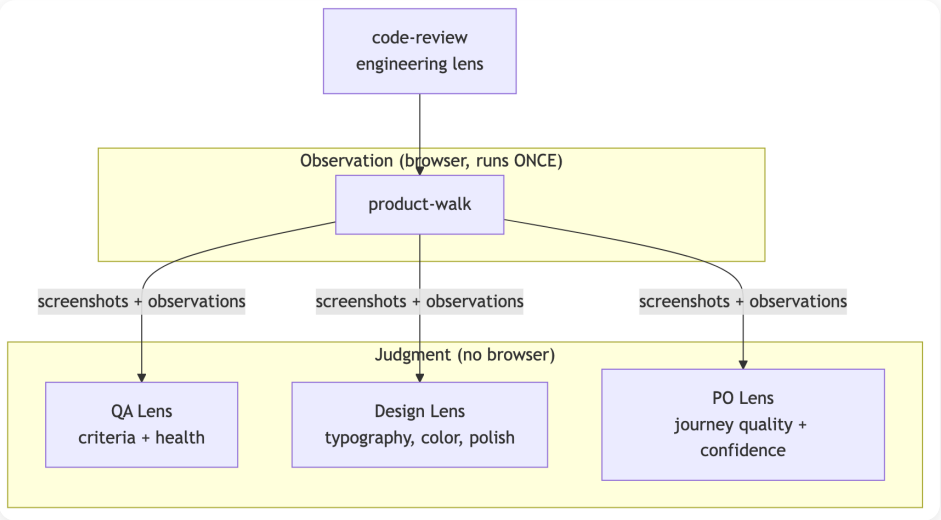
Visual and copy quality

The Design lens scores screenshots across eight categories (typography, color, spacing, layout, components, interaction, content,

polish) and runs an AI slop detector. The Content & Copy sub-check evaluates whether UI text sounds human, is tonally consistent, and matches the product's personality.

The Architecture: Observe Once, Judge Through Lenses

The architecture separates observation from judgment:



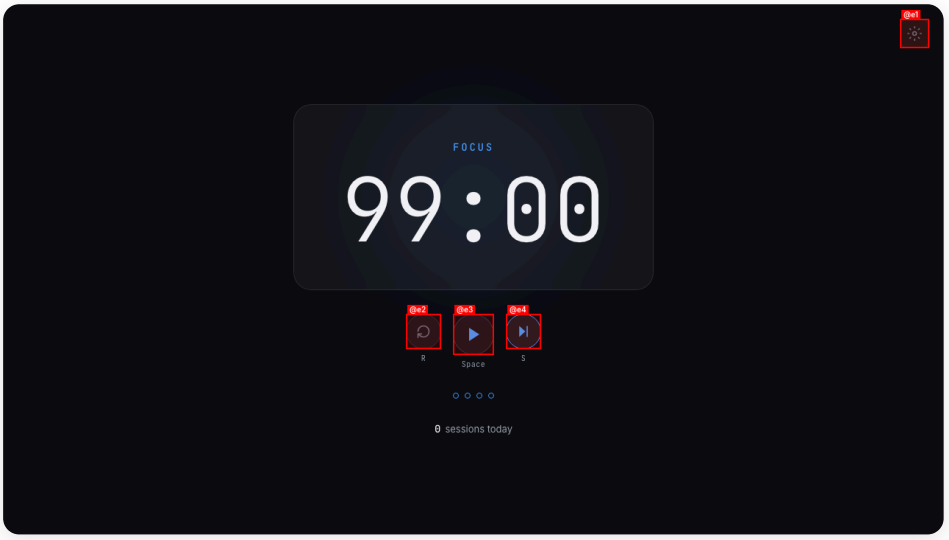
code-review runs first, using only CLI tools (linting, dependency audit, AI code analysis). No browser. It produces an engineering assessment: architecture score, dead code count, security findings.

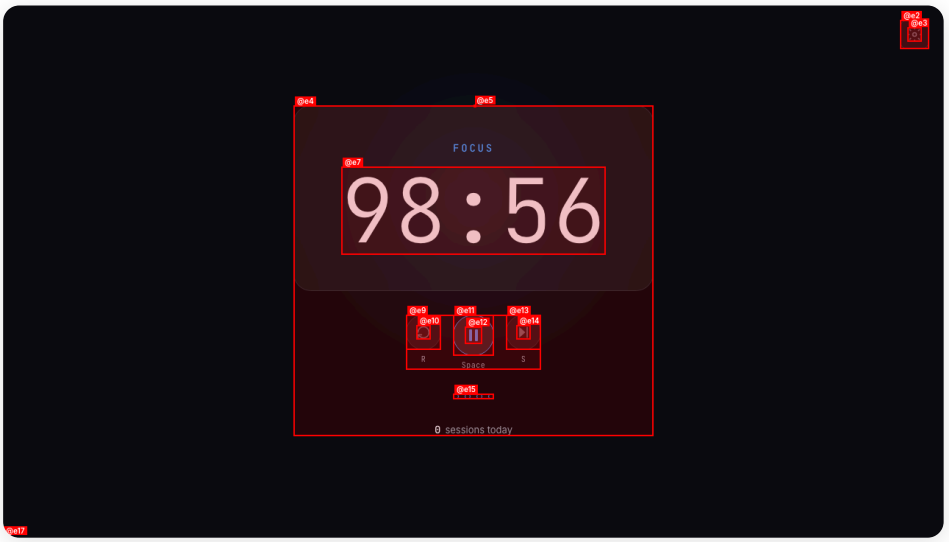
product-walk opens the browser once, navigates every screen state, takes screenshots, and records structured observations. Then the browser closes.

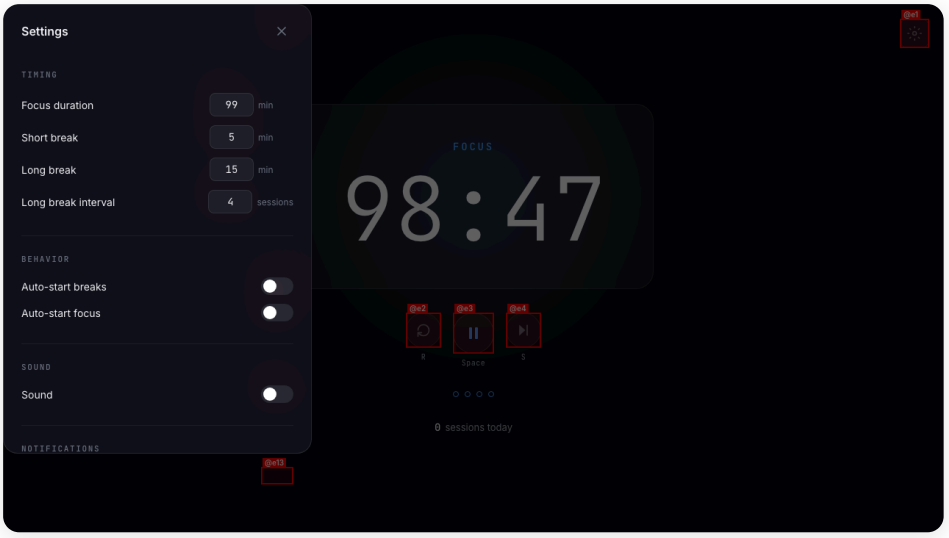
evaluation runs three lenses on the walk data — QA, Design, and PO — without touching a browser. Each lens scores independently. If any lens needs more information, a targeted **re-walk** opens the browser for specific screens only.

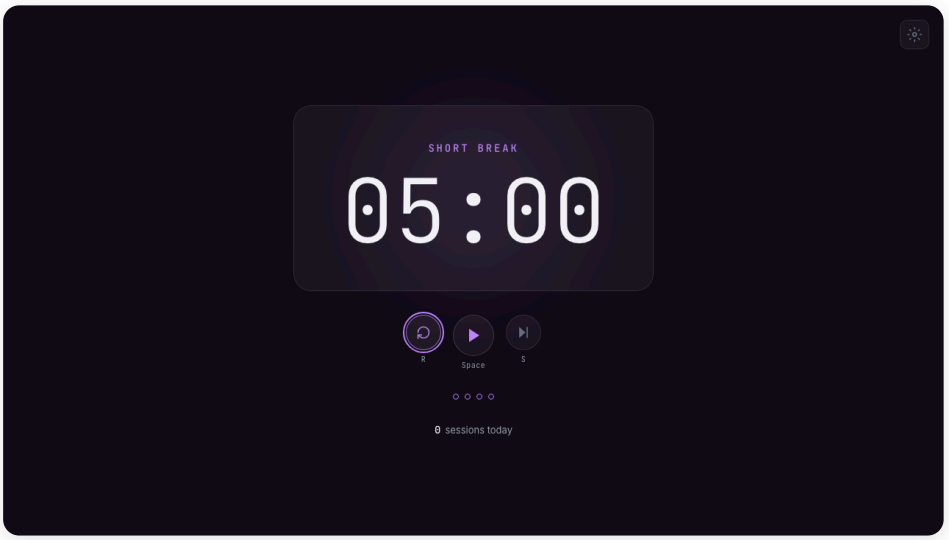
The Final Product

After 5 cycles, Rouge produced a deployed Pomodoro timer that met its vision: atmospheric, opinionated, crafted.









> "Epoch has converged strongly on its vision. The core promise — a focus timer beautiful enough to leave on screen — is delivered through a cohesive techno-futuristic aesthetic, atmospheric phase transitions, and purposeful minimalism."

Key final numbers:

- 90 tests passing

- 67/67 acceptance criteria at 100%
- Health score: 82/100
- Lighthouse: 91 / 100 / 100 / 100
- AI code audit: 90/100
- Design score: 86/100 (AI slop score: 6/100 — the product does not feel AI-generated)
- PO confidence: 0.89
- All 6 user journeys rated "polished"

Cost

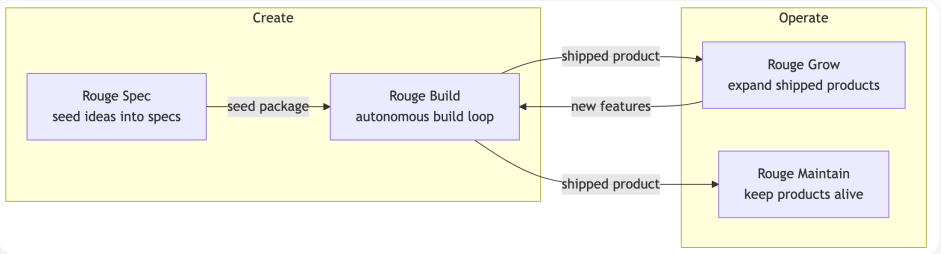
The countdowntimer run consumed approximately **$8.53 of Opus compute time** across 108 minutes of active execution. This includes building, testing, deploying, evaluating, generating change specs, and re-building.

For context: a human developer building this product (design, implementation, testing, accessibility hardening, deployment) would likely spend 1-2 weeks. At typical contractor rates ($100-150/hour), that's $4,000-12,000. Rouge produced a comparable result for under $10 in compute — three orders of magnitude cheaper.

The real cost was in system development: building Rouge itself. But that cost amortizes across every product Rouge builds after the first.

The Rouge Product Line

Building a product from scratch is only one capability. The full vision covers the entire lifecycle:



Rouge Spec — "What to build"

Interactive seeding via Slack. Human + AI co-design a product through conversation with eight discipline-specific personas. Produces a vision, specs, design artifacts, and infrastructure plan.

Rouge Build — "How to build it"

The Karpathy Loop. Takes a seed package and autonomously builds the product through iterative cycles of building, testing, evaluating, and refining. This is what built Epoch.

Rouge Grow — "Make it better"

Feature expansion on shipped products. Unlike Build (which creates from zero), Grow works with existing users, existing data, and existing patterns that must be preserved. It reads analytics (PostHog), user feedback, and market signals to decide what to build next.

Rouge Maintain — "Keep it alive"

Autonomous production upkeep. SBOM and CVE scanning, bug triage from Sentry error streams, dependency updates, database migration management, performance regression detection. No new features — just keeping the lights on and the foundation solid.

What We're Building Toward

- **Module hierarchy**: Large products decompose into modules with dependency-aware build ordering. Rouge has a DAG-based resolver for this.
- **Cross-product learning**: The personal Library layer accumulates intelligence across projects. Product #11 benefits from everything learned building products #1 through #10.
- **Cost predictability**: Before a build starts, the cost estimation engine projects low/mid/high bounds. "This SaaS will cost $50-150 in compute" — then the human decides whether to proceed.
- **Production readiness from day one**: Every Rouge-built product ships with analytics, error monitoring, security headers, CI/CD, legal pages, and i18n support. Not bolted on after — baked into the scaffold.

The goal: a factory where you describe a product over coffee, approve a cost estimate, and come back to a deployed, monitored, production-ready application. Rouge handles everything between the idea and the first user.